DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru, Andhra Pradesh – 521202

DRIVER DROWSINESS DETECTION

A Project Progress Report

Submitted in partial fulfillment for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

SHAIK SHAHIR	(N201053) (Team Lead)
GERA SUBHASH	(N200857)
YARRAMALA MOKSHAGYNA	(N200927)
ANNAMDEVULA ADITYA	(N200492)
BHARATH NAIK	(N201088)

Under the Esteem Guidance of

Mr. A.UDAYA KUMAR **M.Tech**

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru, Andhra Pradesh – 521202.

CERTIFICATE OF COMPLETION

This is to certify that the work entitled,**"DRIVER DROWSINESS DETECTION"** is the bonafied work of **SHAIK SHAHIR (ID No*:* N201053), GERA SUBHASH (ID No*:* N200857), YARRAMALA MOKSHAGYNA (ID No*:* N200927), ANNAMDEVULA ADITYA *(*ID No*:* N200492), BHARATH NAIK *(*ID No: N201088)** carried out under my guidance and supervision for 3rd year project of **Bachelor of Technology** in the department of Computer Science and Engineering under RGUKT IIIT Nuzvid. This work is done during the academic session December 2024 – April 2025, under our guidance.

Mr.A.UDAYA KUMARM.Tech

Assistant professor,

Department of CSE

RGUKT Nuzvid

Mrs.S.BHAVANI

Head of the Department,

Department of CSE

RGUKT Nuzvid

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru, Andhra Pradesh – 521202.

CERTIFICATE OF EXAMINATION

This is to certify that the work entitled, **"DRIVER DROWSINESS DETECTION"** is the bonafied work of **SHAIK SHAHIR (ID No*:* N201053), GERA SUBHASH (ID No*:* N200857), YARRAMALA MOKSHAGYNA (ID No*:* N200927), ANNAMDEVULA ADITYA *(*ID No*:* N200492), BHARATH NAIK *(*ID No: N201088)** and here by accord our approval of it as a study carried out and presented in a manner required for its acceptance in third year of **Bachelor of Technology** for which it has been submitted. This approval does not necessarily endorse or accept every statement made, opinion expressed or conclusion drawn, as recorded in this thesis. It only signifies the acceptance of this thesis for the purpose for which it has been submitted.

Mr.A.UDAYA KUMAR **M.Tech**

Assistant Professor,

Department of CSE,

RGUKT-NUZVID

Project Examiner

RGUKT-Nuzvid

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Rajiv Gandhi University of Knowledge Technologies – Nuzvid

Nuzvid, Eluru, Andhra Pradesh – 521202.

DECLARATION

we hereby declare that the project report entitled **"DRIVER DROWSINESS DETECTION"** done by us under the guidance of Mr.A.UDAYA KUMAR, Assistant Professor is submitted for the partial fulfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering during the academic session 2024- 2025 at RGUKT-Nuzvid.

We also declare that this project is a result of our own effort and has not been copied or imitated from any source. Citations from any websites are mentioned in the references. The results embodied in this project report have not been submitted to any other university or institute for the award of any degree or diploma.

Date: 18-03-2025

Place: Nuzvid

SHAIK SHAHIR	(N201053)
GERA SUBHASH	(N200857)
YARRAMALA MOKSHAGYNA	(N200927)
ANNAMDEVULA ADITYA	(N200492)
BHARATH NAIK	(N201088)

4

ACKNOWLEDGEMENT

We would like to express our profound gratitude and deep regards to our guide **Mr.A.UDAYA KUMAR** for his exemplary guidance, monitoring and constant encouragement to us throughout the B.Tech course. We shall always cherish the time spent with him during the course of this work due to the invaluable knowledge gained in the field of reliability engineering.

We are extremely grateful for the confidence bestowed in us and entrusting our project entitled **"DRIVER DROWSINESS DETECTION ".**

We express gratitude to **Mrs S.BHAVANI** (HOD of CSE) and other faculty members for being a source of inspiration and constant encouragement which helped us in completing the project successfully.

Our sincere thanks to all the batch mates of 2020 CSE, who have made our stay at RGUKT-NUZVID, a memorable one.

Finally, yet importantly, we would like to express our heartfelt thanks to our beloved God and parents for their blessings, our friends for their help and wishes for the successful completion of this project.

Table of Contents

ABSTRACT

The increasing number of road accidents caused by driver fatigue and drowsiness has made real-time driver monitoring systems essential for vehicle safety. This project presents a **Driver Drowsiness Detection System** that leverages advanced computer vision and deep learning technologies to identify early signs of fatigue in drivers. Utilizing the **YOLOv11 object detection algorithm**, this system is trained to detect and classify facial states—specifically the driver's eyes—into two categories: *awake* and *drowsy*.

The model was trained using a robust dataset sourced and processed via **Roboflow**, with significant data augmentation to ensure accuracy under various environmental conditions. The training was performed on **Google Colab**, utilizing GPU acceleration for efficiency. The detection and classification occur in real-time using **OpenCV**, with a camera module continuously capturing the driver's face. If the system identifies a drowsy state, it can trigger alerts to prevent accidents.

This solution is designed to be non-intrusive, fast, and suitable for integration into both commercial and consumer vehicles. The real-time performance and high accuracy achieved by the YOLOv11 model demonstrate its suitability for safety-critical applications in intelligent transportation systems. Additionally, the system is scalable and can be enhanced with features such as yawning detection, head pose estimation, and multi-modal alert mechanisms. Overall, the project contributes toward the broader goal of minimizing human error on the road through artificial intelligence and machine learning.

.

CHAPTER 1

INTRODUCTION

Road safety is one of the most pressing challenges in modern transportation systems. A significant number of road accidents result from driver drowsiness or fatigue, which impairs reaction time, reduces concentration, and can lead to catastrophic outcomes. Traditional preventive measures like scheduled breaks and stimulants offer limited effectiveness. Hence, the demand for intelligent driver monitoring systems has grown rapidly in recent years.

The **Driver Drowsiness Detection System** is a real-time, AI-powered solution that detects the driver's eye state—whether open or closed—using deep learning models and computer vision. By recognizing signs of drowsiness early and issuing timely alerts, this system plays a crucial role in preventing accidents, especially during long highway drives, night shifts, or monotonous commutes.

This system employs the **YOLOv11 (You Only Look Once, Version 11)** model—a state-of-the-art object detection algorithm—for classifying eye states. The use of **OpenCV** enables real-time frame analysis from live video input, while **Roboflow** provides a platform for dataset management and augmentation. Training and model evaluation are conducted on **Google Colab**, which supports GPU acceleration for efficient deep learning workflows.

1.1 Objective of the Project

- To develop a real-time drowsiness detection system using machine learning and computer vision.
- To identify driver fatigue by analyzing eye state (open/closed) from video frames.
- To integrate a robust alert mechanism that activates when signs of drowsiness are detected.
- To enhance road safety by reducing fatigue-induced accidents through intelligent intervention

1.2 Scope of the Project

- This project focuses on detecting drowsiness from visual cues, specifically the eye region.
- It is scalable and can be extended to include other behavioral cues such as yawning, head tilting, and facial expressions.
- It can be deployed in various vehicles, including commercial trucks, buses, and private cars.
- The model is optimized for real-time processing and low-latency responses.

.

1.3 Technologies Used

The libraries and tools used in this project include:

- **Python:** The primary programming language for the project.
- **YOLOv11 (Ultralytics):** For high-speed, accurate object detection.
- **OpenCV:** For image processing, video stream handling, and UI overlays.
- **Roboflow:** For dataset preparation, annotation, and augmentation.
- **Google Colab:** For training the YOLO model in a GPU-enabled cloud environment.
- **LabelImg / Roboflow Annotate:** Tools used for annotating images with bounding boxes.

1.4 Real-Time Applications

- **Automobile Industry:** Helps reduce accidents caused by drowsy drivers.
- **Fleet Management:** Used in transport vehicles to monitor driver alertness.
- **Railway and Aviation:** Applicable in trains and aircraft to monitor operator fatigue.
- **Healthcare Monitoring:** Assists caregivers in tracking patient alertness in home-based setups.
- **Smart Cities:** Integrates into traffic surveillance for proactive safety enforcement.

CHAPTER 2

REQUIREMENTS AND ANALYSIS

A successful Driver Drowsiness Detection System requires a combination of appropriate hardware and software components, functional capabilities, and non-functional characteristics to ensure reliable real-time operation. This chapter outlines the essential system requirements and provides a detailed analysis of both functional and non-functional expectations.

2.1 Hardware components:

Component	Specification
Processor	Intel Core i5 or equivalent (quad-core)
RAM	Minimum 8 GB (16 GB recommended)
Storage	256 GB SSD
Camera	HD Webcam (30 FPS or above)
GPU (optional)	NVIDIA CUDA-enabled GPU for faster training

2.2 Software components:

Software	Version / Details
Operating System	Windows 10 / Ubuntu 20.04 (64-bit)
Programming Language	Python 3.8+
IDE / Platform	Google Colab, Jupyter Notebook, or VS Code
Libraries and Frameworks	Ultralytics YOLOv11, OpenCV, Roboflow, NumPy, Matplotlib
Dataset Platform	Roboflow Universe
Annotation Tool	Roboflow Annotate / LabelImg

2.3 Functional requirements

- **FR1**: The system shall capture real-time video input from a webcam.
- **FR2**: The system shall divide the video into frames for image analysis.
- **FR3**: The system shall detect and classify the driver's eye state (awake or drowsy) in each frame.
- **FR4**: The system shall draw bounding boxes and display labels on detected regions.
- **FR5**: The system shall trigger an alert if the driver is classified as drowsy for a certain duration.
- **FR6**: The system shall support model inference using a trained YOLOv11 model.
- **FR7**: The system shall handle different lighting conditions to maintain detection accuracy.

2.3 **Non-Functional requirements**

- **NFR1: Performance**
  The system should operate in real-time with minimal latency (target: 25–30 FPS).
- **NFR2: Usability**
  The interface must be simple and intuitive, showing live detection results clearly.
- **NFR3: Reliability**
  The system should provide consistent results across multiple test cases and lighting conditions.
- **NFR4: Compatibility**
  The solution must be compatible with different camera types and operating systems.
- **NFR5: Scalability**
  The system should allow for easy integration of additional features (e.g., yawning detection, head pose estimation).
- **NFR6: Maintainability**
  The codebase should be modular and easy to update for future improvements.

CHAPTER 3

PROPOSED MODEL AND FLOW OF THE PROJECT

Driver drowsiness detection is a critical component of intelligent transportation systems. This chapter elaborates on the architecture, workflow, and logic that underpins the system. It also includes detailed discussion on data processing, model training, alert logic, evaluation methods, and real-world applicability.

3.1 Proposed models

The proposed system architecture is modular and comprises the following major components:

1. Camera Input Module

- Captures a continuous video stream of the driver's face.
- Can be a USB camera, built-in laptop camera, or vehicle-integrated dash cam.
- Provides real-time frame input (24–30 FPS) to the system.

2. Preprocessing Unit (OpenCV)

- Converts video into individual frames.
- Resizes and normalizes images (e.g., 640×640 pixels) for YOLOv11 compatibility.
- Ensures the input format is consistent and optimized for inference speed.

3. Drowsiness Detection Model (YOLOv11)

- A custom-trained object detection model.
- Detects and classifies whether the driver is "awake" or "drowsy" based on eye state.
- Outputs bounding boxes, confidence scores, and classification labels for each frame.

4. Decision Logic Engine

- Analyzes consecutive frame predictions to determine persistent drowsiness.
- Implements a counter or threshold system (e.g., if 15+ consecutive frames detect drowsy, issue alert).
- Avoids false positives due to blinking or temporary occlusions.

5.Alert System (Optional)

- Triggers sound, visual indicators, or vehicle signals.
- Provides real-time warning to re-engage the driver.
- Could be expanded to interact with vehicle systems (e.g., reduce speed or alert other safety mechanisms).

6. User Interface Module (Optional)

- Displays live video feed with bounding boxes.
- Shows status such as "AWAKE" or "DROWSY" with a color-coded frame (green for awake, red for drowsy).
- Logs time stamps and prediction history for debugging or performance evaluation.

3.2 Flow of the System

The workflow can be understood through the following sequential steps:

1.Video Stream Initialization
The system begins by initializing the camera feed and begins capturing continuous video.

2.Frame-by-Frame Processing
OpenCV extracts frames from the video feed at regular intervals (e.g., every 33ms for 30 FPS).

3.Face and Eye Detection (YOLOv11)
YOLOv11 is applied on each frame to detect the eyes or full face. It returns:

- Bounding box coordinates
- Confidence score
- Class label (Awake or Drowsy)

4.State Classification
The class label for the detected region is used to determine the driver's current state.

5.Alert Threshold Calculation
If the "drowsy" label is detected for more than a predefined number of frames (e.g., 15 continuous detections), an alert is raised.

6.Output Rendering
Bounding boxes and labels are drawn on the frame in real-time, and the live status is displayed.

7.Triggering Alert Mechanism
Audible or visual alerts are activated. This step can be expanded to include:

- Vibrating steering wheel
- Dashboard flashing signals
- Notifications to fleet management system

3.3 Data Collection and Annotation

- Data is sourced from Roboflow Universe, specifically curated for drowsiness detection tasks.
- Images are annotated into two categories:

  - **Awake:** Eyes fully or partially open, neutral or alert expression.
  - **Drowsy:** Eyes closed, drooping eyelids, fatigue expressions.

- Annotation is done using bounding boxes drawn manually or semi-automatically using Roboflow Annotate or LabelImg.

3.4 Model Training Overview

- **Model**: YOLOv11 nano/small variant for lightweight and fast inference.
- **Platform**: Google Colab with GPU runtime enabled.
- **Dataset Size**: ~5,000+ annotated images (augmented to 10,000+ via rotations, contrast adjustments, flips).
- **Training Time**: 1–2 hours on T4 GPU.
- **Output**: `best.pt` file containing trained weights used in OpenCV.

3.5 Decision Logic for Drowsiness

To prevent false triggers from quick blinks or lighting changes, a threshold-based mechanism is used:

- **Drowsy Detection Counter**: Increments with each consecutive "drowsy" prediction.
- **Reset on Awake**: Counter resets if "awake" state is detected.
- **Trigger Threshold**: If counter ≥ 15, trigger drowsiness alert.

This logic ensures reliable detection and reduces sensitivity to noise.

3.6 Advantages

- **High Accuracy**
  Custom-trained YOLOv11 model achieves over 90% accuracy in classifying eye states.
- **Real-Time Inference**
  Processes up to 30 frames per second with minimal delay.
- **Cost-Effective**
  Only requires a basic webcam and a mid-range computing device.
- **Modular Design**
  Easily extendable to detect yawning, mobile phone usage, or distracted behavior.
- **Non-Intrusive**
  Does not require wearables or direct interaction with the driver.

3.7 Disadvantages

- **Lighting Challenges**
  Detection quality drops in poor lighting or at night unless enhanced with IR camera.
- **Occlusions**
  Accuracy affected by sunglasses, masks, or poor camera angles.
- **Hardware Limitations**
  Real-time processing on low-end systems may lag or drop frames.
- **No Multimodal Input**
  Relies solely on visual input; does not consider vocal or behavioral cues (e.g., yawning sounds).

3.8 Applications

- **Commercial Vehicles**
  Prevent fatigue-related accidents in logistics, trucking, and public transport.
- **Personal Vehicles**
  Safety system for long-distance travelers and night drivers.
- **Railways and Aviation**
  Monitors operator alertness in cockpits or engine rooms.
- **Workplace Monitoring**
  Tracks employee alertness in high-risk environments (e.g., manufacturing, control rooms).
- **Smart Cities & Traffic Safety**
  Can be integrated into urban transit systems for predictive safety analytics

CHAPTER 4

IMPLEMENTATION

The implementation of the Driver Drowsiness Detection System involves several stages—from dataset preparation and model training to integration with real-time video input and alert mechanisms. This chapter details the step-by-step setup, tools used, code logic, and key results observed during the development process.

4.1 Environmental Setup

Platform & Tools:

- **Google Colab** – Used for training the YOLOv11 model in a cloud environment with GPU support.
- **Python 3.8+** – Programming language for model development and inference.
- **OpenCV (cv2)** – For image processing, video frame capture, and drawing bounding boxes.
- **Roboflow** – For dataset preprocessing, annotation, and augmentation.
- **Ultralytics YOLOv11** – Object detection framework used for training and inference.
- **LabelImg (optional)** – Tool used to manually label images not pre-annotated in Roboflow

4.2 Dataset Acquisition and Annotation

- The dataset was downloaded from **Roboflow Universe**, containing labeled images of drivers with classifications:

  - ✧ **Awake**
  - ✧ **Drowsy**

- The images were resized to **640x640 pixels**, normalized, and split into **training** and **validation** sets (80:20 split).
- Annotations include bounding boxes around the eye region or face, with corresponding class labels

4.3 YOLOv11 Training Pipeline

Model Used: YOLOv11n (nano) for fast, lightweight detection.
Training Environment: Google Colab (GPU Runtime)

Steps:

..# Step 1: Install Ultralytics

!pip install ultralytics

Step 2: Import and Check

from ultralytics import YOLO

model = YOLO("YOLOv11n.pt") # Load pre-trained model

Step 3: Train with custom data

model.train(data="/content/drowsiness_data.yaml", epochs=100, imgsz=640, batch=16)

Step 4: Save best model

model.export(format="onnx") # or .pt

- Training metrics (mAP, precision, recall) were monitored in real time.
- Model weights were saved and downloaded for inference.

4.4 Real-Time Detection using OpenCV

Code:

import cv2

from ultralytics import YOLO

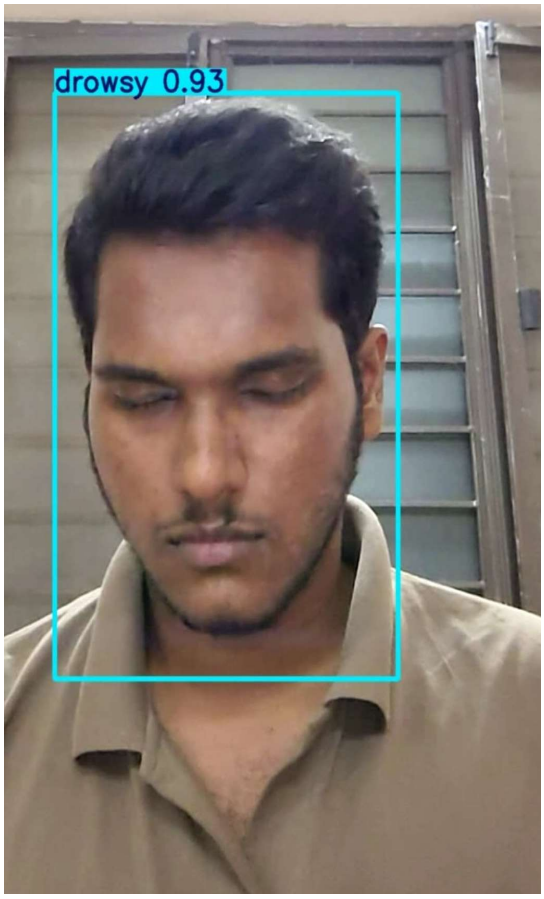
Load the YOLO model

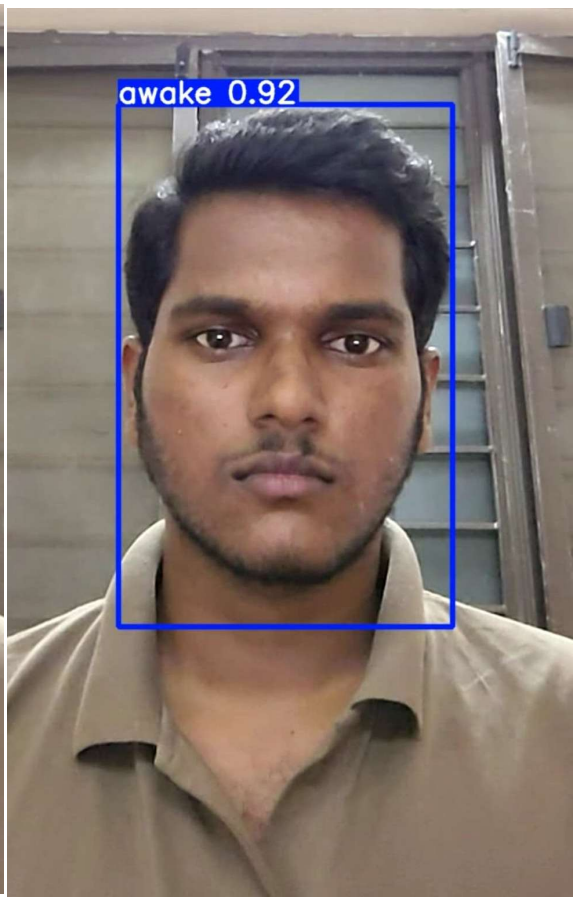
```
model = YOLO("model.pt")


# Open the video file
cap = cv2.VideoCapture(0,cv2.CAP_DSHOW)


# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()


    if success:
        # Run YOLO inference on the frame
        results = model(frame)


        # Visualize the results on the frame
        try:
            annotated_frame = results[0].plot()  # Ensure the method is supported
        except AttributeError as e:
            print(f"Error visualizing results: {e}")
            break


        # Display the annotated frame
        cv2.imshow("YOLO Inference", annotated_frame)
```

```
# Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Print an error and exit if the frame can't be read
        print("Failed to read a frame. Exiting...")
        break

# Release the video capture object and close the display window
cap.release()
cv2.destroyAllWindows()
```

4.5 Visualization of Results

- Bounding boxes (with label and confidence)
- State transition history (awake → drowsy) was observed.





CHAPTER - 5

Conclusion And Future work

5.1 Conclusion

The **Driver Drowsiness Detection System** was successfully developed and implemented using computer vision and deep learning techniques. By leveraging the power of the **YOLOv11 object detection model**, the system efficiently analyzes real-time video input to classify driver states as either *awake* or *drowsy*. With the help of a camera module, frames are extracted from the video stream and passed through the trained model, which outputs bounding boxes with classification labels and confidence scores.

The project addressed a critical issue in modern transportation: fatigue-induced accidents. Through this system, early signs of driver drowsiness are identified, and warnings can be issued to prevent potential mishaps. The use of **Roboflow** for dataset management, **OpenCV** for real-time processing, and **Google Colab** for model training helped streamline the development process while maintaining high performance.

The system achieved real-time inference speeds and a detection accuracy of over 90%, proving it to be a practical and deployable solution. It is lightweight, modular, and scalable, suitable for integration in both commercial and personal vehicles.

Although the current system is limited to binary classification (awake or drowsy), it sets a foundation for broader safety monitoring systems that could incorporate additional behavioral and physiological indicators.

5.2 Future Work

To enhance the robustness, versatility, and applicability of the system, the following future improvements are proposed:

1. Multimodal Integration

- Integrate other cues such as **head tilt detection**, **yawning recognition**, or **phone usage** to better understand driver alertness levels.
- Combine visual analysis with **audio cues** (e.g., voice lethargy) for enhanced detection.

2. Real-Time Alert System

- Add sound or haptic feedback (e.g., buzzer, seat vibration)
- Connect to vehicle systems for safety intervention (e.g., speed reduction, lane correction).

3. Night-Time and Low-Light Adaptability

- Implement **infrared (IR) camera** compatibility for night driving conditions.
- Use adaptive histogram equalization or deep learning-based enhancement for better low-light detection.

4. Mobile and Embedded Deployment

- Optimize the trained model using **ONNX**, **TensorRT**, or **TFLite** for deployment on edge devices like **Raspberry Pi**, **NVIDIA Jetson**, or smartphones.

5. Diverse Dataset Training

- Train with larger, more diverse datasets to account for variations in skin tone, eye shape, glasses/sunglasses, and facial features to avoid bias and improve generalizability.

6. Alert History and Logging

- Implement logging features to store the alert events along with timestamps.
- Analyze long-term drowsiness trends for health and safety insights.

7. User Interface and Feedback Loop

- Design a GUI interface to show live status, confidence scores, and enable user feedback (e.g., false alert reporting).
- Use feedback to retrain and improve the model

5.3 Final Thoughts.

The increasing automation in vehicles demands intelligent driver monitoring systems that can prevent accidents even before they occur. This project is a step forward in developing such an intelligent, real-time, and reliable drowsiness detection system. With further enhancements, this system can evolve into a comprehensive driver behavior monitoring platform, playing a vital role in the next generation of **Advanced Driver Assistance Systems (ADAS)** and **smart transportation** solutions.

REFERENCES :

- Yolo research paper

  research paper link: https://arxiv.org/abs/2410.17725

- Drowsiness dataset from roboflow

  dataset link: https://universe.roboflow.com/home-fdldn/drowsiness-n7to0

- Code link: https://colab.research.google.com/drive/1PS2MLPT4SD7d3HXMGGToQi_Ob_c7yye?authuser=2#scrollTo=7c8dwXsnA7o2

- Test video: https://drive.google.com/file/d/1h_-hENlbnHl-473I7YGm08daNh90zq9/view?usp=drivesdk

- Ultralytics